

GaiaDB Manager原理分享

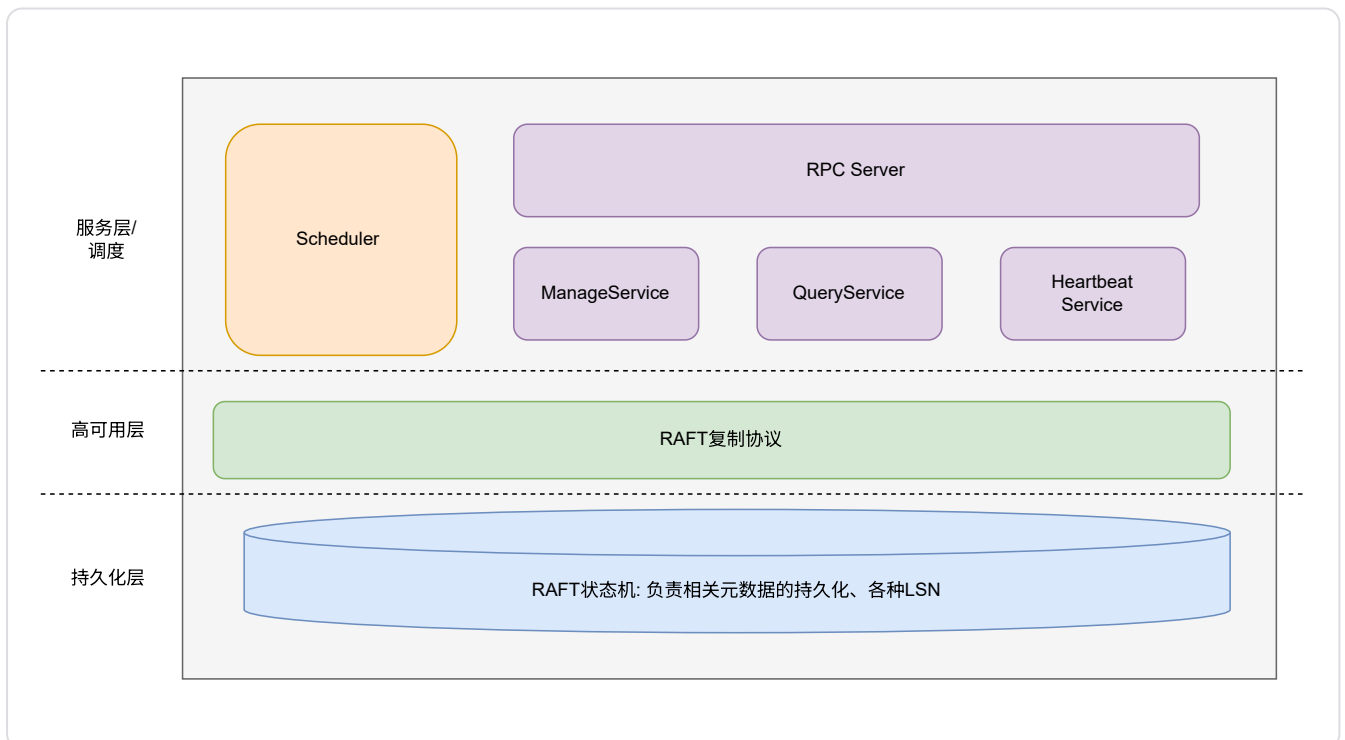
目录

- 一、定位
- 二、内部架构
- 三、主要流程
 - 3.1 常见Manage命令
 - 3.2 Extent Map管理
 - 3.3 集群最低LSN推进
 - 3.4 主从切换

一、定位

GaiaDB Manager定位为GaiaDB实例的中心节点，负责管理数据库实例的元数据，提供计算层逻辑PageNO到物理存储地址的映射，同时基于GaiaDB的特殊架构设计，Manager还兼顾数据库redolog 最低LSN水位的推进。单个GaiaDB数据库实例独占一组Manager实例，一般线上是3个，这3个实例间基于RAFT协议实现高可用与高可靠。

二、内部架构



Manager内部从上到下可大致分成3层：

1. 接口层/调度层
 - a. ManageService 负责接受外部命令，如建表、建segment等
 - b. QueryService提供元数据的查询服务
 - c. HeatbeatService 接收各模块的心跳信息，心跳信息中包含了各模块的LSN信息，该信息是Manager推进集群LSN的重要输入
 - d. Scheduler 负责内部任务的调度:
 - i. 如最重要的周期性推进实例最低LSN, 该最低LSN是LogService做Log GC的最主要输入
 - ii. 另外已删除表的延迟回收也由scheduler负责
2. 高可用层: 这部分是基于RAFT协议实现的，基于厂内braft库开发
3. 持久化层: 体现在raft 快照及raft log中，主要负责元数据的持久化及各模块的LSN

三、主要流程

3.1 常见Manage命令

1. create_space/alter_space/extent_space: 这块逻辑比较简单，主要是space相关元数据操作
2. add_segment/enable_segment/drop_segment:
 - a. add_segment: 创建新的segment，并构建出相应segment replica，注意此时segment 尚未enable，因此segment并不会被分配出去，其上面无任何数据，因此这里创建出来的replica 的recover_status都是RecoveryStatus::RECOVERY_FINISH
 - b. enable_segment: 正式对外暴露segment空间，从此刻开始segment 参与space的空间分配
3. add_replica/drop_replica/drop_page_server: 存量segment 增加/减少 部分副本，drop部分相对简单，add_replica 时因该segment的其他存量副本可能已经有数据了，因此会涉及较复杂的recover流程，recover流程是在page server端实现的，当page server端replica recover 完成时会report 给manager，manager才会在元数据中修改其状态为RecoveryStatus::RECOVERY_FINISH
4. 模块元数据: add_log_service/drop_log_service, add_compute_node/drop_compute_node,.....

3.2 Extent Map管理

Space的逻辑空间由很多Page组成，在GaiaDB里Page大小固定为16K，而64个连续的Page被称为Extent(固定1MB大小)，Extent是GaiaDB里逻辑空间到Segment物理空间的最小映射单位。当Space需要新空间，会请求Manager分配若干个Extent，这些Extent 会均匀的分布到集群带剩余空间的Segments中。Space 逻辑空间到Segment对应物理空间的映射，就是Extent Map存在的根本意义。

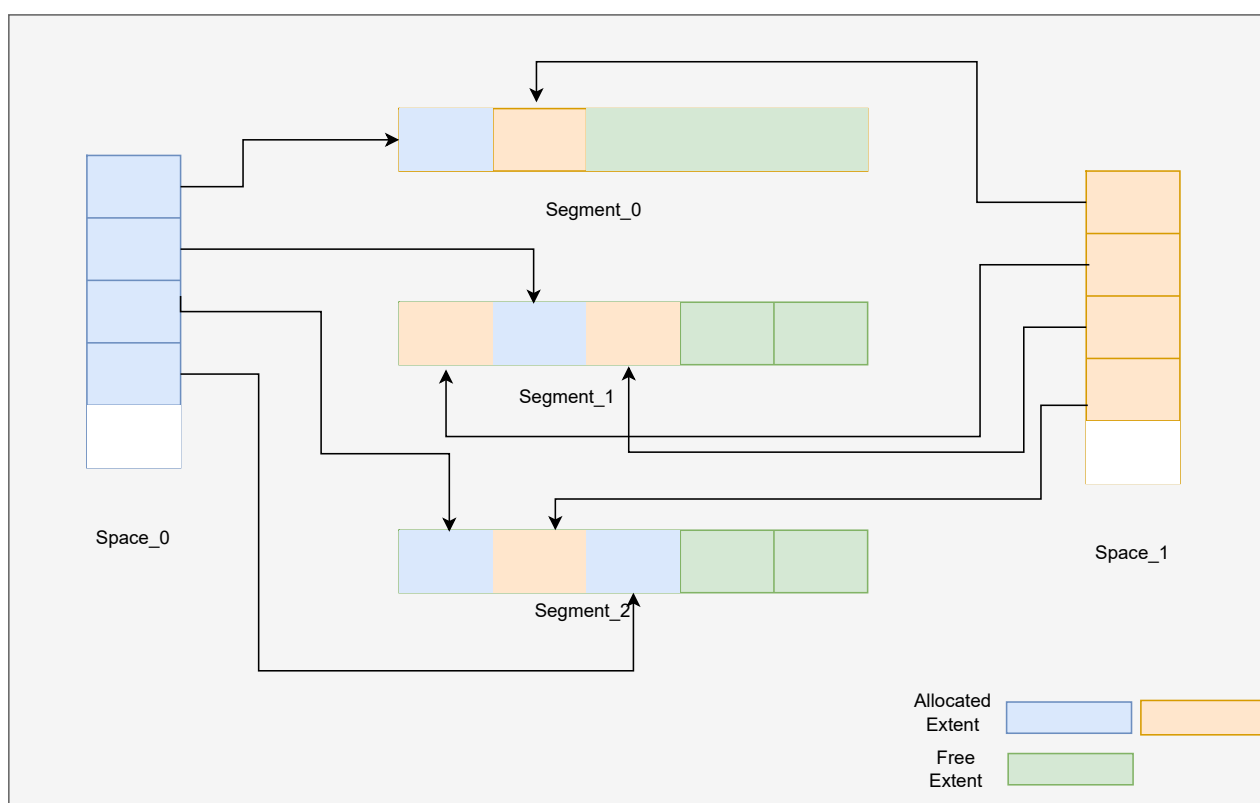
</> 某个物理Extent的数据结构

C++

```

1 struct PhysicalEntry {
2     common::segment_t segment_id;
3     common::extent_t physical_extent_id;
4 };
5
6 class Space {
7     //省略更多细节
8 private:
9     gaiadb::pb::SpaceMeta _space_meta;
10    std::atomic<common::extent_t> _extent_map_size;
11    std::atomic<common::extent_t> _extent_map_used;
12    PhysicalEntry* _extent_map = nullptr; //核心在这里， 其映射关系为: extent_index
    = logical_page_no / 64;
13                                     //其对应物理Extent地址
    为:_extent_map[extent_index];
14 };

```



3.3 集群最低LSN推进

集群最低LSN存在的意义: GaiaDB的计算主节点写Redo log到LogService, LogService提供了容量有限的缓存来存储这些log, PageServer和计算从节点会不断从LogService拉取Log。当某

条Redo Log不再被PageServer/从计算节点需要后，其不再需要保存在LogService缓存中，这就是LogService的GC过程，而这里最低LSN就是LogService GC的参照输入，如 $\text{Log LSN} \leq \text{最低LSN}$ ，这本条Log可GC。

最低LSN推进原理：

1. 各模块通过给Leader Manager的心跳，周期性汇报其最低LSN:如PageServer会汇报其上Segment副本的GC LSN(可理解为写盘LSN)，计算节点汇报Apply LSN
2. Manager 在心跳处理中，会把这些LSN暂存在元数据中
3. Leader Manager的Scheduler 内置一个定时任务，该任务会根据2中缓存的各副本、模块LSN，计算出最新的ps_lowest_lsn、cn_lowest_lsn(可能还有列存最新LSN等)
4. Leader 写raft log，log中包含3计算出来的ps_lowest_lsn、cn_lowest_lsn
5. 各Manager的raft state_machine在Apply本Log时，更新元数据中的ps_lowest_lsn、cn_lowest_lsn。

 读者想想为啥需要步骤4~5?

3.4 主从切换

得益于raft，当Leader Manager故障或挂了，其他Follower Manager会重新当选Leader并继续服务，而集群元数据和集群最低LSN(cn_lowest_lsn、ps_lowest_lsn)的变更都走raft，该部分新Leader能够恢复到与Leader一模一样的状态，**因此集群最低LSN永不后退，但这并不代表各模块、各PS副本的LSN不后退**。这是因为各Segment副本LSN、计算节点LSN等信息是由各模块通过心跳汇报的，而心跳信息未走raft，因此新主这部分信息是缺失的。

各模块或PS Segment副本的LSN短暂后退并不会带来什么问题，这是因为：

1. 新Manager的Scheduler起来时，会强制把计算节点的LSN对齐到集群cn_lowest_lsn、PS Segment副本的LSN对齐到集群ps_lowest_lsn
2. 如果计算节点没挂、存储Page Server没挂，那么下一个心跳周期，模块心跳会带上来最新的LSN，新Manager可据此更新
3. 如果有模块挂了：
 - a. 计算层挂了：没影响，因此计算节点本身就是无状态，从Manager拉到其对应LSN，根据此LSN从Log Service重新拉取Log可恢复
 - b. 存储层Page Server挂了：可能导致部分Page重复Apply Log，但不影响正确性，因为Page Server 的Segment Apply Log是幂等操作